

Abdullah,

MongoDB Capstone Project

Project Title

Business Scenario

A food delivery company wants to analyze:

```
Customers
Restaurants
Orders
Payments
Delivery Partners
Ratings
City-wise performance
Revenue
Delivery delays
```

The goal is to build a MongoDB database using collections and write queries for business analysis.

Database Name

```
use food_delivery_capstone_db
```

Required Collections

```
customers
restaurants
```

orders
delivery_partners

Collection 1: customers

```
db.customers.insertMany([
  {
    customer_id: 1,
    name: "Rahul Sharma",
    city: "Hyderabad",
    membership: "Gold",
    phone: "9876543210"
  },
  {
    customer_id: 2,
    name: "Priya Reddy",
    city: "Bangalore",
    membership: "Silver",
    phone: "9876543211"
  },
  {
    customer_id: 3,
    name: "Amit Kumar",
    city: "Mumbai",
    membership: "Gold",
    phone: null
  },
  {
    customer_id: 4,
    name: "Sneha Patel",
    city: "Chennai",
    membership: "Bronze",
    phone: "9876543213"
  },
  {
    customer_id: 5,
    name: "Arjun Verma",
    city: "Delhi",
    membership: "Silver",
    phone: "9876543214"
  }
])
```

```
}  
])
```

Collection 2: restaurants

```
db.restaurants.insertMany([  
  {  
    restaurant_id: 101,  
    name: "Spice Hub",  
    city: "Hyderabad",  
    cuisine: "Indian",  
    rating: 4.5  
  },  
  {  
    restaurant_id: 102,  
    name: "Pizza Corner",  
    city: "Bangalore",  
    cuisine: "Italian",  
    rating: 4.2  
  },  
  {  
    restaurant_id: 103,  
    name: "Green Bowl",  
    city: "Chennai",  
    cuisine: "Healthy",  
    rating: 4.7  
  },  
  {  
    restaurant_id: 104,  
    name: "Burger Street",  
    city: "Mumbai",  
    cuisine: "Fast Food",  
    rating: 3.9  
  },  
  {  
    restaurant_id: 105,  
    name: "Royal Tandoor",  
    city: "Delhi",  
    cuisine: "Indian",  
    rating: 4.8  
  }  
])
```

```
}  
])
```

Collection 3: delivery_partners

```
db.delivery_partners.insertMany([  
  {  
    partner_id: 201,  
    partner_name: "FastMove Logistics",  
    city: "Hyderabad",  
    rating: 4.4  
  },  
  {  
    partner_id: 202,  
    partner_name: "QuickShip",  
    city: "Bangalore",  
    rating: 4.1  
  },  
  {  
    partner_id: 203,  
    partner_name: "SpeedKart",  
    city: "Mumbai",  
    rating: 4.6  
  },  
  {  
    partner_id: 204,  
    partner_name: "DoorDash India",  
    city: "Delhi",  
    rating: 4.0  
  }  
])
```

Collection 4: orders

```
db.orders.insertMany([  
  {  
    order_id: 1001,
```

```
customer_id: 1,
restaurant_id: 101,
partner_id: 201,
items: [
  { item_name: "Biryani", quantity: 2, price: 250 },
  { item_name: "Kebab", quantity: 1, price: 180 }
],
order_amount: 680,
payment: {
  mode: "UPI",
  status: "Success"
},
order_status: "Delivered",
delivery_time_minutes: 35,
order_rating: 5
},
{
  order_id: 1002,
  customer_id: 2,
  restaurant_id: 102,
  partner_id: 202,
  items: [
    { item_name: "Pizza", quantity: 1, price: 500 },
    { item_name: "Garlic Bread", quantity: 1, price: 150 }
  ],
  order_amount: 650,
  payment: {
    mode: "Card",
    status: "Success"
  },
  order_status: "Delivered",
  delivery_time_minutes: 42,
  order_rating: 4
},
{
  order_id: 1003,
  customer_id: 3,
  restaurant_id: 104,
  partner_id: 203,
  items: [
    { item_name: "Burger", quantity: 2, price: 180 },
    { item_name: "Fries", quantity: 1, price: 120 }
  ],
  order_amount: 480,
```

```
    payment: {
      mode: "Cash",
      status: "Pending"
    },
    order_status: "Pending",
    delivery_time_minutes: null,
    order_rating: null
  },
  {
    order_id: 1004,
    customer_id: 4,
    restaurant_id: 103,
    partner_id: null,
    items: [
      { item_name: "Salad Bowl", quantity: 1, price: 350 }
    ],
    order_amount: 350,
    payment: {
      mode: "UPI",
      status: "Failed"
    },
    order_status: "Cancelled",
    delivery_time_minutes: null,
    order_rating: null
  },
  {
    order_id: 1005,
    customer_id: 5,
    restaurant_id: 105,
    partner_id: 204,
    items: [
      { item_name: "Tandoori Chicken", quantity: 1, price: 600 },
      { item_name: "Naan", quantity: 2, price: 60 }
    ],
    order_amount: 720,
    payment: {
      mode: "UPI",
      status: "Success"
    },
    order_status: "Delivered",
    delivery_time_minutes: 50,
    order_rating: 5
  },
  {
```

```
    order_id: 1006,
    customer_id: 1,
    restaurant_id: 101,
    partner_id: 201,
    items: [
      { item_name: "Paneer Curry", quantity: 1, price: 300 },
      { item_name: "Roti", quantity: 4, price: 25 }
    ],
    order_amount: 400,
    payment: {
      mode: "Card",
      status: "Success"
    },
    order_status: "Delivered",
    delivery_time_minutes: 30,
    order_rating: 4
  }
])
```

Capstone Tasks

Part 1: Basic MongoDB Queries

1. Display all customers.
 2. Display all restaurants.
 3. Display only customer name, city and membership.
 4. Find all customers from Hyderabad.
 5. Find all Gold members.
 6. Find restaurants with rating greater than 4.5.
 7. Find orders with amount greater than ₹500.
 8. Find delivered orders.
 9. Find cancelled orders.
 10. Find customers where phone is null.
-

Part 2: Operators

11. Find orders where amount is between ₹400 and ₹700.
12. Find customers from Hyderabad, Delhi or Mumbai.

13. Find restaurants whose cuisine is Indian or Fast Food.
 14. Find orders where payment status is not Success.
 15. Find orders where delivery time is null.
 16. Find orders where rating is greater than or equal to 4.
 17. Find restaurants not located in Bangalore or Chennai.
-

Part 3: Array Queries

18. Find orders containing item `Biryani`.
 19. Find orders containing item `Pizza`.
 20. Find orders where any item quantity is greater than 1.
 21. Find orders where item price is greater than ₹300.
 22. Display only order ID and items.
-

Part 4: Sorting and Limit

23. Sort restaurants by rating descending.
 24. Display top 3 highest rated restaurants.
 25. Sort orders by order amount descending.
 26. Display top 2 highest value orders.
 27. Sort delivery partners by rating descending.
-

Part 5: Update Operations

28. Update customer 1 membership to `Platinum`.
29. Update restaurant 104 rating to `4.1`.
30. Update order 1003 status to `Delivered`.
31. Set delivery time of order 1003 to `45`.
32. Add field `active: true` to all customers.
33. Remove field `active` from all customers.
34. Add a new item to order 1006:

```
{ item_name: "Curd Rice", quantity: 1, price: 120 }
```

Part 6: Delete Operations

- 35. Delete cancelled orders.
 - 36. Delete restaurants with rating less than 4.0.
-

Part 7: Count and Distinct

- 37. Count total customers.
 - 38. Count total orders.
 - 39. Count delivered orders.
 - 40. Count failed payments.
 - 41. Display distinct customer cities.
 - 42. Display distinct restaurant cuisines.
 - 43. Display distinct payment modes.
-

Part 8: Aggregation

44. Revenue by Payment Mode

```
Payment Mode
Total Revenue
```

45. Revenue by Order Status

```
Order Status
Total Revenue
```

46. Average Delivery Time

Calculate average delivery time for delivered orders.

47. Orders by Customer

```
Customer ID
Total Orders
Total Amount
```

48. Orders by Restaurant

```
Restaurant ID
Total Orders
Total Revenue
```

49. Average Rating by Restaurant

```
Restaurant ID
Average Order Rating
```

50. High Revenue Customers

Find customers whose total spending is greater than ₹700.

Part 9: Lookup / Join Style Queries

Use `$lookup`.

51. Orders with Customer Details

Display:

```
Order ID
Customer Name
City
Order Amount
Order Status
```

52. Orders with Restaurant Details

Display:

Order ID
Restaurant Name
Cuisine
Order Amount

53. Orders with Delivery Partner Details

Display:

Order ID
Partner Name
Delivery Time
Order Status

54. Full Order Report

Display:

Order ID
Customer Name
Restaurant Name
Cuisine
Partner Name
Order Amount
Payment Mode
Payment Status
Order Status
Delivery Time
Rating
